Practice Interview

Objective

The partner assignment aims to provide participants with the opportunity to practice coding in an interview context. You will analyze your partner's Assignment 1. Moreover, code reviews are common practice in a software development team. This assignment should give you a taste of the code review process.

Group Size

Each group should have 2 people. You will be assigned a partner

Part 1:

You and your partner must share each other's Assignment 1 submission.

Part 2:

Create a Jupyter Notebook, create 6 of the following headings, and complete the following for your partner's assignment 1:

- Paraphrase the problem in your own words.

In [ ]:
```
# Check if there are duplicates in a tree. If there's one duplicate, return
# If there are more than one, return the value of the duplicate closest to t
```

- Create 1 new example that demonstrates you understand the problem. Trace/walkthrough 1 example that your partner made and explain it.

In [ ]:
```
# Using Level Order Traversal
# Input: [1,2,3,4,4,5,6]
# Output: 4

# Partner's Example
# Input: [1,3,5,7,9,11]
# Output: -1
```

```
# In this example, there are no duplicate values and thus the output is -1.
```

- Copy the solution your partner wrote.

In [ ]:
```python
class TreeNode(object):
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

from collections import deque

def is_duplicate(root: TreeNode) -> int:
    if not root:
        return -1

    # Using a set to keep track of seen values
    seen = set()
    queue = deque([root])

    # Level-order traversal using BFS
    while queue:
        node = queue.popleft()

        # Check for duplicates
        if node.val in seen:
            return node.val
        seen.add(node.val)

        # Add left and right children to the queue
        if node.left:
            queue.append(node.left)
        if node.right:
            queue.append(node.right)

    # No duplicates found
    return -1
```

- Explain why their solution works in your own words.

In [ ]:
```
# Use level order traversal so that when a duplicate is detected, it will be
# Create variable seen to store node values. Create queue variable as needed
# When visiting a node, remove it from the queue. Then check if its value is
# and continue traversing tree in order based on variable queue. If no dupli
```

- Explain the problem's time and space complexity in your own words.

In [ ]:
```
# Time complexity: We traverse each node only once - O(n)
# Space Complexity: Since values are stored in a variable and at most n valu
```

- Critique your partner's solution, including explanation, and if there is anything that should be adjusted.

In [ ]:
```
# Solutions is good. My only minor comment is that the code "queue = deque([
# "#Level-order traversal using BFS" as the queue is part of the level order
```

Part 3:

Please write a 200 word reflection documenting your process from assignment 1, and your presentation and review experience with your partner at the bottom of the Jupyter Notebook under a new heading "Reflection." Again, export this Notebook as pdf.

Reflection

In [ ]:
```
# I am not new to coding, but I have only limited experience with Python, pa
# basic concept of recursion, I struggle to write recursive functions entire
# find examples or blueprints. These resources give me a framework to start
# Once I've modified the code, I run it, observe its behavior, and debug it
# code until a get more comfortable writing it from scratch.

# When it comes to reviewing my partner's code, I took a similar approach. I
# I consult online materials to ensure the logic and implementation are corr
# more comfortable with Python and recursive functions as I do more of these
```

Evaluation Criteria

We are looking for the similar points as Assignment 1

- Problem is accurately stated
- New example is correct and easily understandable
- Correctness, time, and space complexity of the coding solution
- Clarity in explaining why the solution works, its time and space complexity
- Quality of critique of your partner's assignment, if necessary

Submission Information

🚨 **Please review our Assignment Submission Guide** 🚨 for detailed instructions on how to format, branch, and submit your work. Following these guidelines is crucial for your submissions to be evaluated correctly.

Submission Parameters:

- Submission Due Date: `HH:MM AM/PM - DD/MM/YYYY`
- The branch name for your repo should be: `assignment-2`
- What to submit for this assignment:
  - This Jupyter Notebook (assignment_2.ipynb) should be populated and should be the only change in your pull request.
- What the pull request link should look like for this assignment: `https://github.com/<your_github_username>/algorithms_and_data_struc`
  - Open a private window in your browser. Copy and paste the link to your pull request into the address bar. Make sure you can see your pull request properly. This helps the technical facilitator and learning support staff review your submission easily.

Checklist:

- [ ] Created a branch with the correct naming convention.
- [ ] Ensured that the repository is public.
- [ ] Reviewed the PR description guidelines and adhered to them.
- [ ] Verify that the link is accessible in a private browser window.

If you encounter any difficulties or have questions, please don't hesitate to reach out to our team via our Slack at `#cohort-3-help`. Our Technical Facilitators and Learning Support staff are here to help you navigate any challenges.